

Dispositivos Conectados

Modos de consumo de energia reduzido

2025/2026

Authors: Paulo C. Bartolomeu

1 Objetivos

- Familiarização com os modos de operação do ESP32-C6;
- Utilização dos modos de consumo *light-sleep* e *deep-sleep*;
- Utilização dos modos de *wake-up* através do *timer* do RTC, GPIOs e sinal EXT1.

2 Introdução

O ESP32-C6, tal como outros microcontroladores da família ESP da Espressif, integra um conjunto lato de modos de gestão de energia concebidos para otimizar o equilíbrio entre o desempenho computacional e a eficiência energética. Esta funcionalidade é particularmente relevante em aplicações alimentadas por bateria, dispositivos IoT (*Internet of Things*), sistemas de monitorização ambiental, sensores remotos e equipamentos portáteis, onde a autonomia e o consumo reduzido são fatores críticos.

O chip foi projetado com uma arquitetura de baixo consumo baseada num núcleo RISC-V de 32 bits, complementado por diversos elementos (p. ex., periféricos de comunicação) e por um sistema de regulação dinâmica de energia que permite desligar seletivamente blocos internos quando não estão em uso. Desta forma, o ESP32-C6 consegue adaptar-se continuamente aos requisitos da aplicação, alternando entre modos de plena operação e modos de *deep-sleep* com consumos da ordem de poucos microamperes. Esta flexibilidade permite que o microcontrolador opere em diferentes modos de energia, como *Active*, *Modem-sleep*, *Light-sleep* e *Deep-sleep*, ajustando o nível de atividade dos componentes internos (CPU, rádio Wi-Fi 6, Bluetooth LE 5.3, *Real-Time Clock*, memória e periféricos).

O resultado é uma plataforma versátil que consegue, por exemplo, operar em modo ativo durante transmissões Wi-Fi de alta velocidade e, logo de seguida, entrar num estado de “sono profundo” (*deep-sleep*) com consumos inferiores a 10 μ A, preservando dados essenciais na memória *Real-Time Clock* (RTC). Essa capacidade torna o ESP32-C6 especialmente adequado para dispositivos de recolha de dados, sistemas de controlo distribuído, módulos de telemetria e aplicações industriais de longa duração, em que é necessário combinar conectividade avançada com máxima eficiência energética.

3 Modos de baixo consumo

O ESP32-C6 suporta 5 modos de baixo consumo que permitem ao *developer* uma grande flexibilidade na forma de gerir o seu funcionamento, permitindo uma grande variedade de opções de *tradeoff* entre desempenho e optimização do uso de energia.

- **Modo ativo** (*Full power*): todos os periféricos e interfaces, como SPI, I2C, Bluetooth, Wi-Fi e ambos os núcleos da CPU, estão ativos. Este modo garante o desempenho máximo, mas também o consumo mais elevado, podendo atingir 260 mA durante picos de transmissão Wi-Fi e cerca de 120 mA em emissão contínua.
- **Modem-sleep**: o processador principal continua ativo, mas o módulo de rádio (Wi-Fi/Bluetooth) entra em suspensão. Permite poupanças significativas de energia, com consumos entre 3 mA e 20 mA, consoante a frequência do processador e as tarefas em execução. Este modo é útil quando o dispositivo processa dados localmente sem necessidade de comunicação de rede contínua.
- **Light-sleep**: o CPU é colocado em pausa, mantendo o conteúdo da RAM e o estado interno dos periféricos principais. As interfaces de comunicação sem fios são desligadas e a alimentação das secções digitais é reduzida. O consumo desce para cerca de 0.8 mA, com tempos de despertar rápidos (microsegundos). É indicado para períodos curtos de inatividade em que é importante preservar o contexto da aplicação.
- **Deep-sleep**: o CPU e quase todos os periféricos são desligados. Mantêm-se apenas ativos o RTC (*Real-Time Clock*) e, opcionalmente, o coprocessador ULP (*Ultra-Low Power*). Este coprocessador, que pode ser programado em *assembly*, permite realizar tarefas simples como ler sensores via I2C ou ADC e acordar o sistema quando um valor ultrapassa um limite. O consumo situa-se entre 150 uA (com ULP ativo) e 10-25 uA, podendo descer até 2.5 uA em modo de hibernação, quando apenas o relógio RTC permanece ativo.
- **Hibernação**: modo de funcionamento que oferece o consumo mais baixo; toda a lógica digital e memória são desligadas, restando apenas o RTC, o que permite que o sistema acorde em intervalos longos pré-definidos. Em aplicações com baterias, este modo pode garantir autonomia superior a um ano, dependendo da frequência de despertar.

O ESP32-C6 integra um conjunto versátil de *wakeup sources* que permitem reativar o sistema a partir dos diferentes modos de *sleep* de forma eficiente e configurável. Estas fontes são fundamentais para otimizar o equilíbrio entre autonomia energética e capacidade de resposta, possibilitando que o microcontrolador permaneça inativo durante longos períodos, acordando apenas quando necessário para executar tarefas específicas.

As fontes de despertar podem ser temporizadas, externas ou internas, e são geridas pelo controlador *Real-Time Clock* (RTC), que continua ativo mesmo quando os vários elementos do *chip* se m desligado. Entre as principais opções *wakeup* de destacam-se as seguintes:

- **Temporizador RTC**: permite acordar o chip após um intervalo de tempo pré-definido, ideal para aplicações periódicas como recolha de dados ou envio de medições.
- **GPIO externo (EXT1)**: possibilita o despertar do sistema quando um ou mais pinos digitais mudam de estado lógico (nível alto ou baixo), funcionando como uma interrupção de *hardware*.
- **ULP Coprocessor**: pode executar tarefas simples enquanto o chip está em sono profundo, como ler sensores I2C ou valores da ADC, e acordar o sistema quando uma condição é atingida.
- **Sensor tátil (Touch)**: em versões compatíveis, permite que o toque num pino capacitivo ative o chip, útil em interfaces humanas de baixo consumo.
- **UART** (apenas Light-sleep): o chip pode acordar ao detetar atividade numa linha série RX,

sendo adequado para módulos GPS, modems ou sensores seriais.

A escolha da fonte de despertar depende do modo de sono utilizado e dos requisitos da aplicação. Em *light-sleep*, o despertar é quase imediato e o estado do sistema é preservado; em *deep-sleep*, o *chip* realiza um reinício controlado, mas pode recuperar dados previamente armazenados na memória RTC.

Nesta aula iremos explorar a utilização dos modos de *light-sleep* e *deep-sleep*.

4 Trabalho Prático

O trabalho prático desta aula desenvolve-se com base na [API de Referência](#) os modos de *sleep* do [ESP32-C6](#) juntamente com o [ESP32-C6 Technical Reference Manual](#).

4.1 Modo ativo

No modo ativo todos os periféricos e interfaces estão ativos, assegurando a resposta mais rápida mas também acarretando o maior consumo.

NOTA IMPORTANTE: Nesta secção o objetivo é apenas o aluno familiarizar-se com o consumo associado ao funcionamento do circuito da Aula 5 no modo ativo através dos gráficos fornecidos. Não é necessário montar o circuito nem programar o ESP32-C6.

As Figuras 1 e 2 ilustram o consumo de corrente registado com o *Nordic Power Profiler Kit II* para a aplicação desenvolvida na Aula 5 de leitura de informação do sensor DHT20 e escrita no cartão microSD (circuito ilustrado na Figura 3).

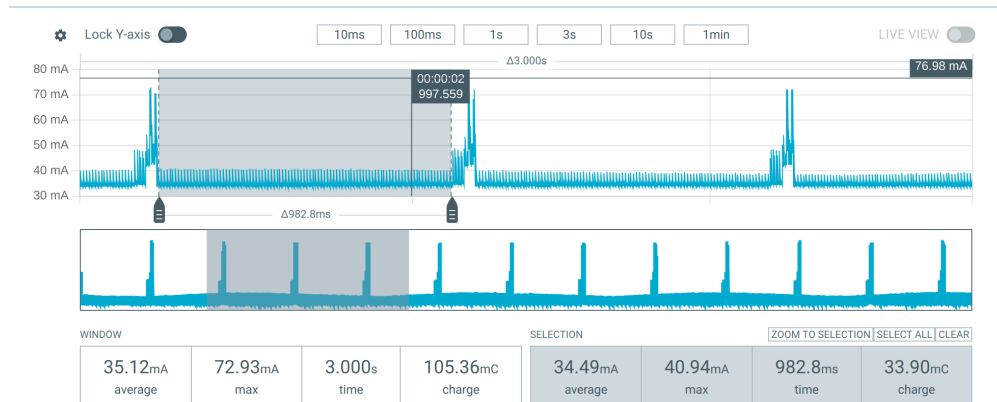


Figura 1: Consumo em modo normal

Como se pode observar, o consumo médio em “repouso” é de 34.49 mA com picos de até 40.94 mA (Figura 1) enquanto que durante o período de leitura/escrita é de 42.16 mA com picos de até 72.16 mA (Figura 2). É também fácil de constatar que o período de amostragem destas variáveis é concordante com a questão da Aula 5, ou seja, um período de 1 segundo entre amostragens sucessivas.

Apesar dos periféricos que mais consomem energia (Wi-Fi, Bluetooth, IEEE 802.15.4) não estarem ainda ativados, o consumo de energia em repouso nesta aplicação é já considerável e potencialmente incompatível com aplicações que tenham requisitos de elevada autonomia sem recurso a uma

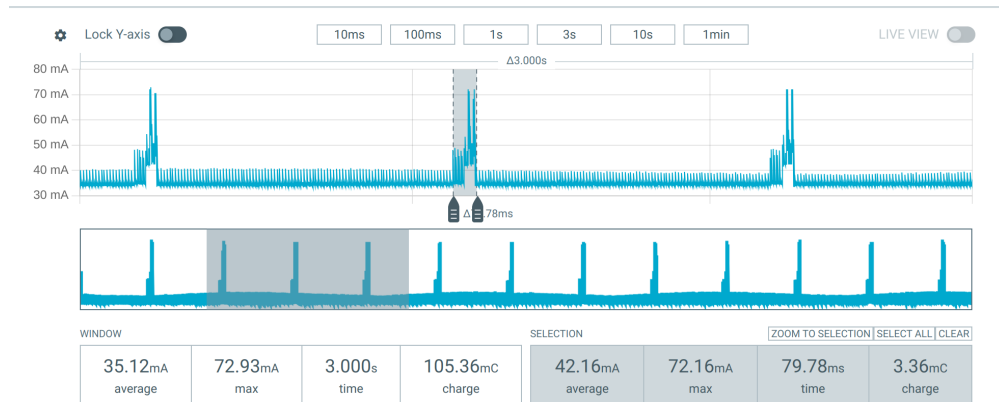


Figura 2: Consumo em modo de leitura de humidade e temperatura e escrita no cartão micro SD

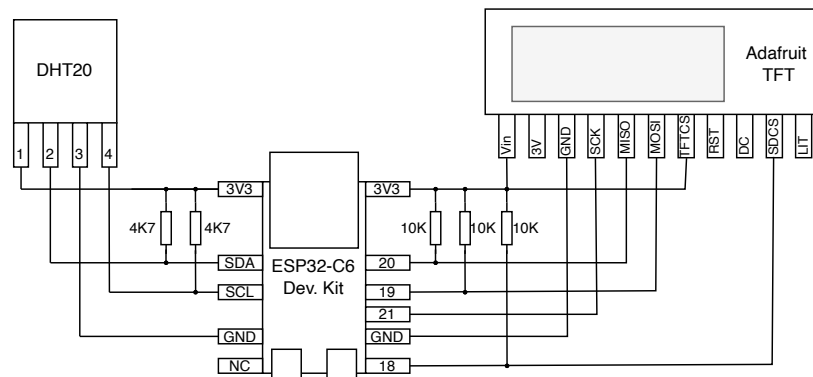


Figura 3: Circuito para integração do leitor de cartões com o sensor DHT20

fonte de energia permanente. A próxima secção endereça este problema apresentando o modo de funcionamento *light-sleep*.

4.2 Modo *light-sleep*

O modo de funcionamento *light-sleep* beneficia de uma redução dramática do consumo do microcontrolador e de tempos de *wake-up* muito curtos, na ordem dos microsegundos, potenciando o seu uso em aplicações em que o tempo de resposta é crítico.

ALERTA IMPORTANTE 1! Nos vários exemplos que vão ser estudados relativamente ao uso dos modos de baixo consumo, *opte pela programação do microcontrolador através da UART disponível na ficha USB-C alternativa* (enumerada tipicamente como `/dev/ttyUSB0`). Quando o ESP32-C6 entra nos modos de *deep-sleep* ou *light-sleep*, o SoC desativa ou desliga os subsistemas dos quais depende a interface USB/JTAG integrada na placa o que, na prática, desconecta o *chip* do computador. Uma vez que a porta UART alternativa é habilitada através de um conversor UART-USB externo, que não depende da alimentação do SoC, a entrada nos modos de *deep-sleep* ou *light-sleep* não tem efeito.

ALERTA IMPORTANTE 2! Os gráficos apresentados foram obtidos por um equipamento (*Nordic Power Profiler Kit II*) que não está disponível para os alunos. Estes gráficos servem apenas como referência do consumo nos vários modos de funcionamento.

4.2.1 Utilização do timer como fonte de *wake-up*

Nesta parte pretende-se colocar o microcontrolador em modo *light-sleep* e acordá-lo periodicamente através do *timer* do controlador RTC (*Real-Time Clock timer*). Para o efeito, descarregue o ficheiro `ls_wktimer.zip` e descompacte-o no seu diretório de trabalho.

Neste programa iremos configurar o *timer* interno do RTC para gerar um evento de *wake-up*. Durante o modo de *light-sleep* o CPU, a maioria dos periféricos e o oscilador principal serão suspensos. O domínio RTC (*Real-Time Clock*) permanece ativo com o seu oscilador lento, que é usado pelo *timer* RTC para manter a contagem do tempo e emitir o sinal que acorda o processador quando o período definido termina.

QUESTÃO 1: Abra o projeto no VSCode e estude o ficheiro `main.c`. Qual a duração do período de *light-sleep* configurado?

QUESTÃO 2: Proceda com a compilação do programa e descarregue-o para o SoC, abrindo de seguida a consola correspondente. Deverá observar no terminal um *output* semelhante ao seguinte:

```
I (259) main_task: Started on CPU0
I (259) main_task: Calling app_main()
going for a nap
napped for 999
going for a nap
napped for 999
going for a nap
napped for 999
```

QUESTÃO 3: Altere o programa, configurando diferentes intervalos de tempo entre *wake-ups* e verifique o respetivo funcionamento.

Usando o programa original testado na Questão 2 e o *Nordic Power Profiler Kit II* mediu-se o consumo de corrente nas diferentes fases do respetivo funcionamento, estando isso ilustrado nas Figuras 4 e 5. Como se pode observar, durante o período de 988 ms o consumo médio é de 239.19 μA com picos de até 259.83 μA (Figura 4). No modo normal, o consumo aumenta substancialmente, para um valor médio 26.25 mA com picos de até 34.39 mA (Figura 5).



Figura 4: Consumo em modo *light-sleep*

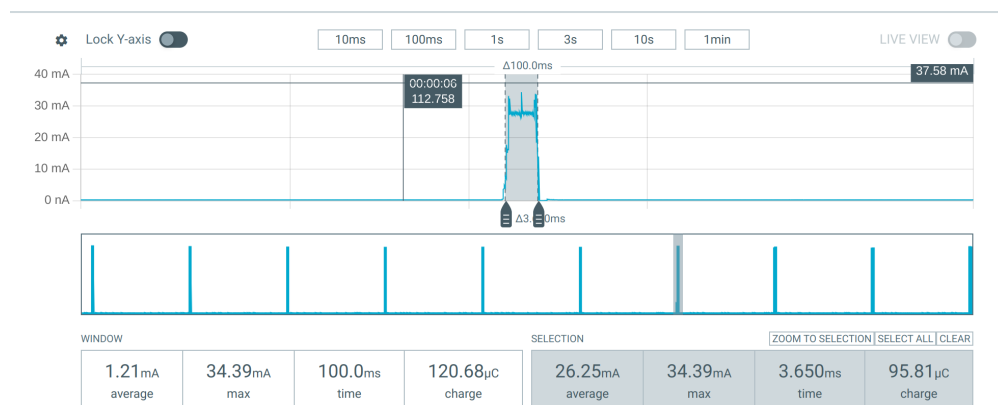


Figura 5: Consumo em modo normal

QUESTÃO 4: Consulte a documentação relativa aos *Sleep Modes* e justifique a invocação da função `console_flush()` no fim do ciclo da função `main`.

4.2.2 Utilização de GPIO como fonte de *wake-up*

Uma fonte alternativa de *wake-up* do modo *light-sleep* é a utilização de GPIOs. Nesta parte iremos usar um botão de pressão, configurado como na Aula 3 (*pulled-up*) e ligado ao GPIO0. Para o efeito, descarregue o ficheiro `1s_wkgpio.zip` e descompacte-o no seu diretório de trabalho.

QUESTÃO 5: Abra o projeto no VSCode e estude o ficheiro `main.c`. Explique qual a função do seguinte bloco de código:

```
if (gpio_get_level(INPUT_PIN) == 0) {
    printf("please release button\n");
    console_flush();
}
```

```

do {
    vTaskDelay(pdMS_TO_TICKS(10));
} while (gpio_get_level(INPUT_PIN) == 0);
}

```

QUESTÃO 6: Proceda com a compilação do programa e descarregue-o para o SoC, abrindo de seguida a consola correspondente. Ao premir o botão de pressão, deverá acordar o SoC do modo *light-sleep* e observar no terminal um *output* semelhante ao seguinte:

```

I (252) main_task: Started on CPU0
I (252) main_task: Calling app_main()
going for a nap
woke up after 2165 ms, reason: button
please release button
going for a nap
woke up after 695 ms, reason: button
please release button
going for a nap
woke up after 144 ms, reason: button
please release button
going for a nap
woke up after 957 ms, reason: button
please release button
going for a nap

```

No modo *light-sleep*, o despertar por *timer* é interno e consome menos energia, ocorrendo após um intervalo de tempo definido, enquanto o despertar por GPIO é externo e reativo, dependente da mudança de nível num pino, exigindo manter parte da lógica de entrada ativa, o que aumenta o consumo.

Tal como apresentado para o *wake-up* via *timer*, mediu-se o consumo de corrente nas diferentes fases do funcionamento com *wake-up* via GPIO, sendo isso documentado nas Figuras 6, 7 e 8. Como se pode observar nas Figuras 6 e 7, o consumo não vai além dos 181.6 μA no modo *light-sleep*, ficando-se em média pelos 160.55 e 161.14 μA , respetivamente. Observe-se também nestas figuras que os intervalos de tempo medidos 687.1 μs e 136.0 μs são muito próximos daqueles reportados na consola (695 μs e 144 μs , respetivamente). No modo normal, o consumo aumenta para um valor médio 21.81 mA com picos de até 34.99 mA (Figura 8).



Figura 6: Consumo em modo *light-sleep* (*wake-up* de GPIO) 1



Figura 7: Consumo em modo *light-sleep* (*wake-up* de GPIO) 2



Figura 8: Consumo em modo normal

4.3 Modo *deep-sleep*

O modo de funcionamento *deep-sleep* permite atingir o mínimo consumo energético possível do microcontrolador, desligando completamente o processador, a RAM principal e a maioria dos periféricos, mantendo apenas o domínio RTC (*Real-Time Clock*) ativo para funções essenciais, como contagem de tempo, retenção de variáveis ou detecção de eventos de despertar. Neste estado, o consumo desce para a ordem dos micro a dezenas de microamperes, tornando-o ideal para aplicações alimentadas por bateria ou de monitorização remota em que os períodos de inatividade são longos. O retorno à operação ativa implica um reinício completo do sistema, com tempos de *wake-up* superiores aos do *light-sleep*, mas aceitáveis em cenários em que a prioridade é maximizar a autonomia energética em detrimento da resposta imediata.

4.3.1 Utilização do timer como fonte de *wake-up*

Nesta parte pretende-se colocar o microcontrolador em modo *deep-sleep* e acordá-lo periodicamente através do *timer* do controlador RTC (*Real-Time Clock timer*). Para o efeito, descarregue o ficheiro `ds_wktimer.zip` e descompacte-o no seu diretório de trabalho.

QUESTÃO 7: Abra o projeto no VSCode e estude o ficheiro `main.c`. Qual o funcionamento esperado para este programa?

QUESTÃO 8: Proceda com a compilação do programa e descarregue-o para o SoC, abrindo de seguida a consola correspondente. Deveria observar no terminal um *output* semelhante ao seguinte, em que a variável de contagem é incrementada a cada ciclo. Contudo isso não ocorre. Porquê? **Pista:** Consulte novamente a documentação relativa aos *Sleep Modes*.

```
I (251) main_task: Started on CPU0
I (251) main_task: Calling app_main()
going to sleep. woken up 0
ESP-ROM:esp32c6-20220919
Build:Sep 19 2022
rst:0x5 (SLEEP_WAKEUP),boot:0x6f (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DIO, clock div:2
load:0x40875730,len:0x1744
load:0x4086b910,len:0xe78
load:0x4086e610,len:0x31c8
entry 0x4086b91a
I (24) boot: ESP-IDF v5.5.1-dirty 2nd stage bootloader
...
I (232) coexist: coex firmware version: b0bcc39
I (251) coexist: coexist rom version 5b8dcfa
I (251) main_task: Started on CPU0
I (251) main_task: Calling app_main()
going to sleep. woken up 1
ESP-ROM:esp32c6-20220919
...
I (251) main_task: Started on CPU0
I (251) main_task: Calling app_main()
going to sleep. woken up 2
```

QUESTÃO 9: Altere o programa de modo a que a variável de contagem seja preservada entre *wake-ups* sucessivos e teste o seu programa.

Usando o setup anteriormente identificado, mediu-se o consumo de corrente no modo *deep-sleep* para esta aplicação com *wake-up* via *timer* do RTC. Como se pode observar nas Figura 9, o consumo

em modo deep-sleep cai para os 8.26 μA em média, atingindo no máximo 10 μA . Em modo normal o consumo é em média de 30.62 mA e no máximo de 50.04 mA (Figura 10).

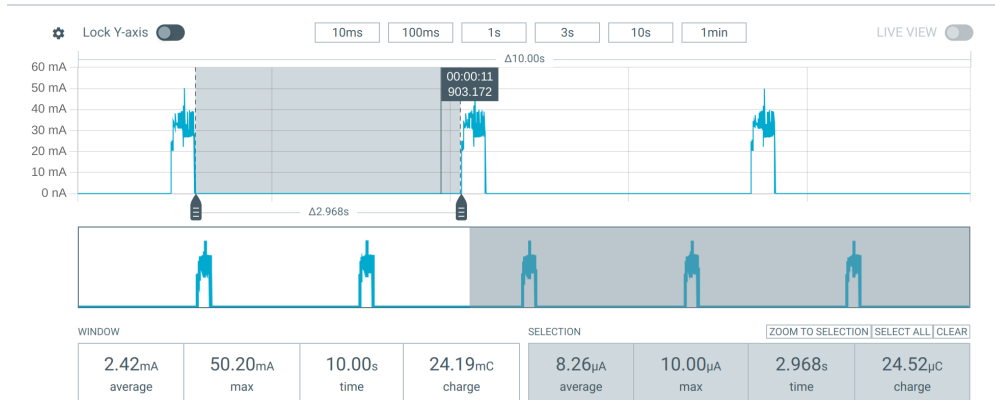


Figura 9: Consumo em modo *deep-sleep* (*wake-up* através do *timer* RTC)

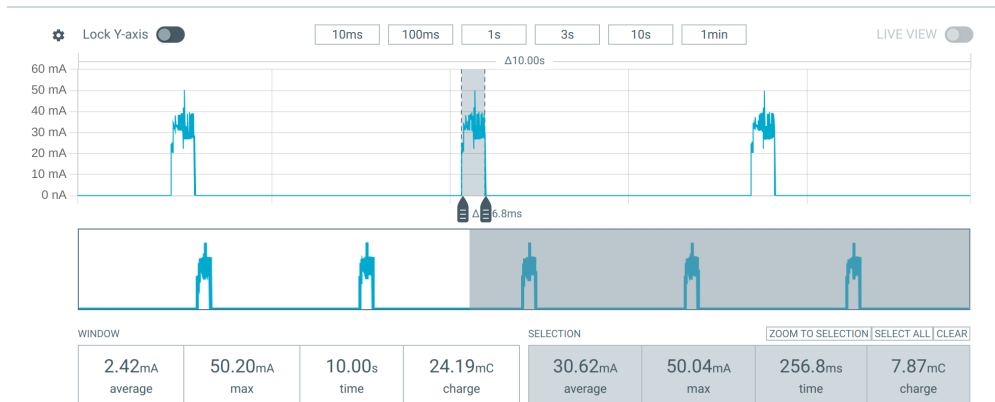


Figura 10: Consumo em modo normal

4.3.2 Utilização do sinal EXT1 como fonte de *wake-up*

O despertar do modo *deep-sleep* através do sinal EXT1 permite que o ESP32-C6 acorde em resposta a um evento externo detetado num ou mais pinos de entrada/saída (GPIO) configurados para esse fim. Este mecanismo é gerido pelo controlador RTC, que permanece parcialmente ativo durante o sono profundo e monitoriza o estado elétrico dos pinos seleccionados, mesmo quando o restante sistema está totalmente desligado.

Ao configurar o EXT1 *wake-up*, é possível definir se o microcontrolador deve acordar quando qualquer dos pinos atinge um determinado nível lógico (ANY_HIGH ou ANY_LOW), ou apenas quando todos os pinos cumprem a condição especificada. Esta flexibilidade permite implementar estratégias de ativação seletiva, como acordar o sistema quando um botão é premido, um sensor altera o seu estado, ou um sinal externo de controlo é recebido.

Nesta parte iremos usar o botão de pressão anteriormente montado para o wake-up do estado de light-sleep (ligado ao GPIO0). Para o efeito, descarregue o ficheiro `ds_wkext1.zip` e descompacte-o no seu diretório de trabalho.

QUESTÃO 10: Abra o projeto no VSCode e estude o ficheiro `main.c`. Compile o programa e teste-o na placa premindo o botão de pressão. O resultado deverá ser o seguinte:

```
I (259) main_task: Started on CPU0
I (259) main_task: Calling app_main()
Going to deep sleep. Woken up 0
ESP-ROM:esp32c6-20220919
Build:Sep 19 2022
rst:0x5 (SLEEP_WAKEUP),boot:0x6f (SPI_FAST_FLASH_BOOT)
SPIWP:0xee
mode:DI0, clock div:2
load:0x40875730,len:0x1744
load:0x4086b910,len:0xe78
load:0x4086e610,len:0x31c8
entry 0x4086b91a
I (24) boot: ESP-IDF v5.5.1-dirty 2nd stage bootloader
...
I (259) main_task: Started on CPU0
I (259) main_task: Calling app_main()
Going to deep sleep. Woken up 1
ESP-ROM:esp32c6-20220919
...
I (259) main_task: Started on CPU0
I (259) main_task: Calling app_main()
Going to deep sleep. Woken up 2
ESP-ROM:esp32c6-20220919
```

Através do *Nordic Power Profiler Kit II* mediu-se o consumo de corrente nas diferentes fases do respetivo funcionamento, tendo-se obtido os resultados documentados nas Figuras 11 e 12. Como se pode observar, o modo *deep-sleep* apresenta um consumo médio de 9.64 μA com um valor máximo de 11.98 μA (Figura 11). Em linha com o que foi anteriormente medido para o *wake-up* via *timer* do RTC, o consumo em modo normal é de 30.79 mA em média e 50.36 mA de máximo.

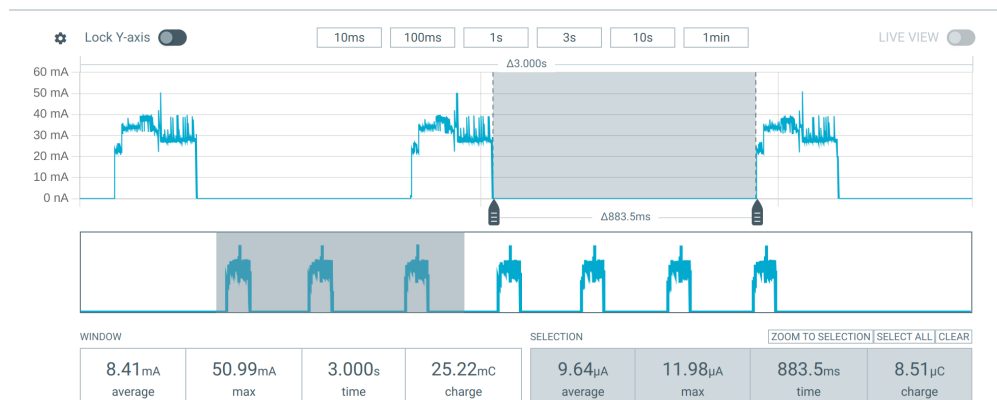


Figura 11: Consumo em modo *deep-sleep* (*wake-up* através de sinal EXT1)

QUESTÃO 11: Monte um segundo botão de pressão na placa com *pull-up* e altere o programa de modo a que, apenas quando ambos os botões forem premidos em simultâneo, o programa acorda do modo *deep-sleep*.

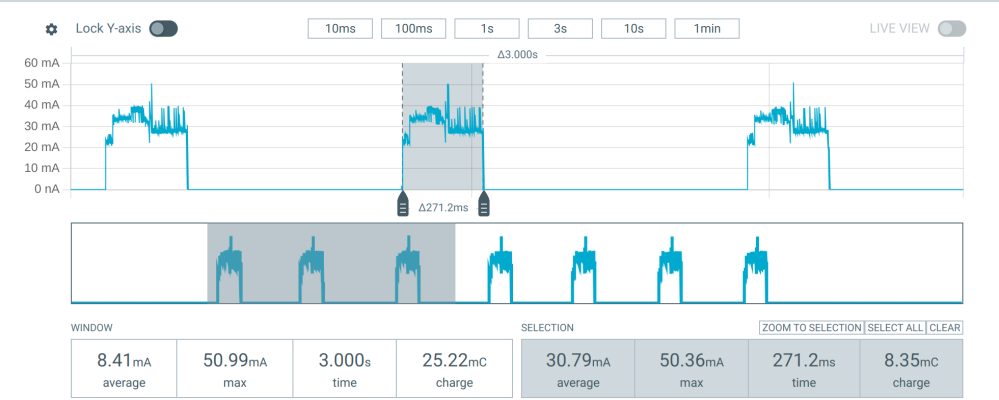


Figura 12: Consumo em modo normal

5 Acknowledgements

Originally authored by [Paulo C. Bartolomeu](#)